1. Basic Query Structure

SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column ASC|DESC
LIMIT number;

Execution Order:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

---

2. SELECT

SELECT * FROM employees;

SELECT name, salary FROM employees;

SELECT DISTINCT department FROM employees;

---

3. WHERE Conditions

SELECT * FROM employees WHERE salary > 50000;

SELECT * FROM employees WHERE department = 'IT';

SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;

SELECT * FROM employees WHERE department IN ('HR','IT');

SELECT * FROM employees WHERE name LIKE 'A%';

Patterns:

'A%' Starts with A
'%A' Ends with A
'%A%' Contains A
'_' Single character

---

4. ORDER BY

SELECT * FROM employees ORDER BY salary DESC;

SELECT * FROM employees ORDER BY department ASC, salary DESC;

---

5. LIMIT / TOP

SELECT * FROM employees LIMIT 5; -- MySQL/PostgreSQL

SELECT TOP 5 * FROM employees; -- SQL Server

---

6. Aggregate Functions

SELECT COUNT(*) FROM employees;

SELECT SUM(salary) FROM employees;

SELECT AVG(salary) FROM employees;

SELECT MIN(salary), MAX(salary) FROM employees;

---

7. GROUP BY

SELECT department, COUNT(*)
FROM employees
GROUP BY department;

---

8. HAVING

SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;

---

9. INSERT

```
INSERT INTO employees(id, name, salary)
VALUES (1, 'John', 50000);
```

Multiple rows:

```
INSERT INTO employees(name, salary)
VALUES
('A', 10000),
('B', 20000);
```

---

10. UPDATE

```
UPDATE employees
SET salary = 60000
WHERE id = 1;
```

---

11. DELETE

```
DELETE FROM employees
WHERE id = 1;
```

Delete all rows:

```
DELETE FROM employees;
```

---

12. CREATE TABLE

```
CREATE TABLE employees (
id INT PRIMARY KEY,
name VARCHAR(50),
salary DECIMAL(10,2),
department VARCHAR(30)
);
```

---

13. ALTER TABLE

```
ALTER TABLE employees
ADD email VARCHAR(100);

ALTER TABLE employees
DROP COLUMN email;

ALTER TABLE employees
MODIFY salary INT;
```

---

14. DROP / TRUNCATE

```
DROP TABLE employees; -- removes structure + data

TRUNCATE TABLE employees; -- removes all rows fast
```

---

15. Constraints

PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT

Example:

```
CREATE TABLE students (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT CHECK(age >= 18),
city VARCHAR(30) DEFAULT 'Delhi'
);
```

---

16. JOINS

INNER JOIN

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.id;
```

LEFT JOIN

```
SELECT *
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id;
```

RIGHT JOIN

```
SELECT *
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.id;
```

FULL JOIN

```
SELECT *
FROM employees e
FULL JOIN departments d
ON e.dept_id = d.id;
```

---

17. SELF JOIN

```
SELECT a.name, b.name AS manager
FROM employees a
JOIN employees b
ON a.manager_id = b.id;
```

---

18. UNION

```
SELECT name FROM students
UNION
SELECT name FROM teachers;
```

Keep duplicates:

```
UNION ALL
```

---

19. Subqueries

```
SELECT name
FROM employees
WHERE salary > (
SELECT AVG(salary) FROM employees
);
```

---

20. EXISTS

```
SELECT name
FROM employees e
WHERE EXISTS (
SELECT 1
FROM departments d
WHERE d.id = e.dept_id
);
```

---

21. CASE

```
SELECT name,
CASE
WHEN salary > 50000 THEN 'High'
WHEN salary > 30000 THEN 'Medium'
ELSE 'Low'
END AS level
FROM employees;
```

---

22. Index

```
CREATE INDEX idx_name
ON employees(name);
```

---

23. Views

```
CREATE VIEW high_salary AS
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Use:

```
SELECT * FROM high_salary;
```

---

24. Transactions

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;

ROLLBACK;
```

---

25. Common Interview Queries

Second Highest Salary

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Duplicate Records

```
SELECT name, COUNT(*)
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

Top 3 Salaries

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

---

26. Normalization

1NF = Atomic values

2NF = Remove partial dependency

3NF = Remove transitive dependency

BCNF = Stronger 3NF

---

27. Wildly Important Tips

Always use WHERE in UPDATE/DELETE

Use indexes on searched columns

Use JOIN instead of nested queries often

Use LIMIT while testing DELETE

NULL != 0

COUNT(*) counts rows

COUNT(column) ignores NULL

---

28. NULL Handling

SELECT * FROM employees WHERE salary IS NULL;

SELECT * FROM employees WHERE salary IS NOT NULL;

SELECT COALESCE(salary,0) FROM employees;

---

29. Window Functions

SELECT name, salary,
RANK() OVER(ORDER BY salary DESC) AS rnk
FROM employees;

ROW_NUMBER()
RANK()
DENSE_RANK()
LEAD()
LAG()

---

30. Must Remember

DELETE = remove rows
TRUNCATE = remove all rows fast
DROP = remove table

WHERE = before grouping
HAVING = after grouping

---

31. Fast Revision (1 Minute)

SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT

JOIN
UNION
SUBQUERY
INDEX
VIEW
TRANSACTION

---

32. If Preparing for Placement

Focus hard on:

SELECT
WHERE
JOINS
GROUP BY
HAVING
Subqueries
Second highest salary
Duplicates
Window Functions

Normalization basics

Indexes